

```
!pip install -q opencv-python numpy matplotlib pandas
```

```
# =====
# IMAGE REPRESENTATION STUDY
# Conversion of Image into Grayscale and Binary Representations
# Edge Detection + ORB Feature Extraction + Comparison
# =====

# =====
# 1. IMPORT LIBRARIES
# =====

import cv2
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from google.colab import files
from IPython.display import display

print("=" * 80)
print("                IMAGE REPRESENTATION STUDY")
print("=" * 80)

# =====
# 2. UPLOAD IMAGE
# =====

print("\nUpload your input image:")

uploaded = files.upload()

image_name = list(uploaded.keys())[0]

# Read original image
original = cv2.imread(image_name)

if original is None:
    raise ValueError("ERROR: Image could not be loaded.")

print("\nImage loaded successfully!")
print("Image Name :", image_name)
print("Image Size :", original.shape)

# =====
# 3. CONVERT BGR TO RGB FOR DISPLAY
# =====

original_rgb = cv2.cvtColor(
    original,
    cv2.COLOR_BGR2RGB
)

# =====
# 4. GRAYSCALE CONVERSION
# =====

gray = cv2.cvtColor(
    original,
    cv2.COLOR_BGR2GRAY
)

# Grayscale conversion formula:
#
# Gray = 0.299R + 0.587G + 0.114B
#

# =====
# 5. BINARY CONVERSION
# =====
```

Enal
noti
to g
exec

```

THRESHOLD = 128

_, binary = cv2.threshold(
    gray,
    THRESHOLD,
    255,
    cv2.THRESH_BINARY
)

# =====
# 6. DISPLAY ORIGINAL, GRAYSCALE AND BINARY
# =====

plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)

plt.imshow(original_rgb)

plt.title(
    "a) Original Color Image",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(
    gray,
    cmap="gray"
)

plt.title(
    "b) Grayscale Image",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.subplot(1, 3, 3)

plt.imshow(
    binary,
    cmap="gray"
)

plt.title(
    f"c) Binary Image (T = {THRESHOLD})",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.suptitle(
    "Image Representation Conversion",
    fontsize=20,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

# =====
# 7. CANNY EDGE DETECTION
# =====

CANNY_LOW = 50

```

Enal
noti
to g
exec

```

CANNY_HIGH = 150

# Edge detection on original color image
# Convert internally to grayscale before Canny

original_for_canny = cv2.cvtColor(
    original,
    cv2.COLOR_BGR2GRAY
)

edge_original = cv2.Canny(
    original_for_canny,
    CANNY_LOW,
    CANNY_HIGH
)

# Edge detection on grayscale

edge_gray = cv2.Canny(
    gray,
    CANNY_LOW,
    CANNY_HIGH
)

# Edge detection on binary

edge_binary = cv2.Canny(
    binary,
    CANNY_LOW,
    CANNY_HIGH
)

# =====
# 8. EDGE DENSITY CALCULATION
# =====

def calculate_edge_density(edge_image):

    edge_pixels = np.count_nonzero(edge_image)

    total_pixels = edge_image.shape[0] * edge_image.shape[1]

    density = (
        edge_pixels / total_pixels
    ) * 100

    return density

edge_density_original = calculate_edge_density(
    edge_original
)

edge_density_gray = calculate_edge_density(
    edge_gray
)

edge_density_binary = calculate_edge_density(
    edge_binary
)

# =====
# 9. DISPLAY EDGE DETECTION RESULTS
# =====

plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)

plt.imshow(
    edge_original,
    cmap="gray"
)

```

Enal
noti
to g
exec

```

cmap= gray
)

plt.title(
    "d) Edge - Original\n(Canny)",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(
    edge_gray,
    cmap="gray"
)

plt.title(
    "e) Edge - Grayscale\n(Canny)",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.subplot(1, 3, 3)

plt.imshow(
    edge_binary,
    cmap="gray"
)

plt.title(
    "f) Edge - Binary\n(Canny)",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.suptitle(
    "Canny Edge Detection Comparison",
    fontsize=20,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

# =====
# 10. ORB FEATURE DETECTOR
# =====

ORB_FEATURES = 1000
ORB_SCALE_FACTOR = 1.2
ORB_LEVELS = 8

orb = cv2.ORB_create(
    nfeatures=ORB_FEATURES,
    scaleFactor=ORB_SCALE_FACTOR,
    nlevels=ORB_LEVELS
)

# =====
# 11. ORB FEATURE EXTRACTION
# =====

# Original image
# ORB requires a single-channel image,
# so original color image is converted to grayscale internally.

```

Enal
noti
to g
exec

```

kp_original, des_original = orb.detectAndCompute(
    original_for_canny,
    None
)

# Grayscale image

kp_gray, des_gray = orb.detectAndCompute(
    gray,
    None
)

# Binary image

kp_binary, des_binary = orb.detectAndCompute(
    binary,
    None
)

# =====
# 12. KEYPOINT COUNTS
# =====

num_original = len(kp_original)

num_gray = len(kp_gray)

num_binary = len(kp_binary)

print("\n")
print("=" * 80)
print("                                ORB FEATURE EXTRACTION")
print("=" * 80)

print(
    f"\nOriginal Color Image : {num_original} keypoints"
)

print(
    f"Grayscale Image      : {num_gray} keypoints"
)

print(
    f"Binary Image         : {num_binary} keypoints"
)

# =====
# 13. DRAW ORB KEYPOINTS
# =====

orb_original_img = cv2.drawKeypoints(
    original_rgb,
    kp_original,
    None,
    color=(0, 255, 0),
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

orb_gray_img = cv2.drawKeypoints(
    gray,
    kp_gray,
    None,
    color=(0, 255, 0),
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

orb_binary_img = cv2.drawKeypoints(
    binary,
    kp_binary,
    None,

```

Enal
noti
to g
exec

```

        color=(0, 255, 0),
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )

# =====
# 14. DISPLAY ORB KEYPOINTS
# =====

plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)

plt.imshow(
    orb_original_img
)

plt.title(
    f"g) ORB Keypoints - Original\n"
    f"No. of Keypoints: {num_original}",
    fontsize=13,
    fontweight="bold"
)

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(
    orb_gray_img,
    cmap="gray"
)

plt.title(
    f"h) ORB Keypoints - Grayscale\n"
    f"No. of Keypoints: {num_gray}",
    fontsize=13,
    fontweight="bold"
)

plt.axis("off")

plt.subplot(1, 3, 3)

plt.imshow(
    orb_binary_img,
    cmap="gray"
)

plt.title(
    f"i) ORB Keypoints - Binary\n"
    f"No. of Keypoints: {num_binary}",
    fontsize=13,
    fontweight="bold"
)

plt.axis("off")

plt.suptitle(
    "ORB Feature Extraction",
    fontsize=20,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

# =====
# 15. FEATURE EXTRACTION SUMMARY
# =====

print("\n")
print("=" * 80)

```

Enal
noti
to g
exec

```

print("                FEATURE EXTRACTION SUMMARY")
print("=" * 80)

print(
    "\nOriginal Color Image:"
)

print(
    f"   ORB Keypoints = {num_original}"
)

print(
    "\nGrayscale Image:"
)

print(
    f"   ORB Keypoints = {num_gray}"
)

print(
    "\nBinary Image:"
)

print(
    f"   ORB Keypoints = {num_binary}"
)

# =====
# 16. COMPARISON TABLE
# =====

comparison_data = {

    "Representation": [
        "Original Color",
        "Grayscale",
        f"Binary (T={THRESHOLD})"
    ],

    "Image Type": [
        "Color (BGR)",
        "8-bit Gray",
        "Binary (0/1)"
    ],

    "Edge Density (%)": [
        round(edge_density_original, 2),
        round(edge_density_gray, 2),
        round(edge_density_binary, 2)
    ],

    "No. of Keypoints (ORB)": [
        num_original,
        num_gray,
        num_binary
    ],

    "Computation Complexity": [
        "High",
        "Medium",
        "Low"
    ],

    "Performance": [
        "High (Rich Info)",
        "Good (Balanced)",
        "Moderate (Less Info)"
    ]
}

df = pd.DataFrame(
    comparison_data
)

```

Enal
noti
to g
exec

```

print("\n")
print("=" * 100)
print("                                COMPARISON TABLE")
print("=" * 100)

display(df)

# =====
# 17. KEYPOINT COMPARISON GRAPH
# =====

representations = [
    "Original\nColor",
    "Grayscale",
    "Binary\n(T=128)"
]

keypoint_values = [
    num_original,
    num_gray,
    num_binary
]

plt.figure(figsize=(10, 7))

bars = plt.bar(
    representations,
    keypoint_values,
    color=[
        "royalblue",
        "seagreen",
        "tomato"
    ],
    edgecolor="black"
)

plt.title(
    "No. of Keypoints (ORB) Comparison",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel(
    "Image Representation",
    fontsize=13
)

plt.ylabel(
    "Number of Keypoints",
    fontsize=13
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.4
)

# Display numbers above bars
for bar, value in zip(
    bars,
    keypoint_values
):
    plt.text(
        bar.get_x() +
        bar.get_width() / 2,
        bar.get_height() + 10,
        str(value),
        ha="center",
        va="bottom"
    )

```

Enal
noti
to g
exec


```

        va="bottom",
        fontsize=13,
        fontweight="bold"
    )

plt.tight_layout()
plt.show()

# =====
# 18. EDGE DENSITY GRAPH
# =====

edge_values = [
    edge_density_original,
    edge_density_gray,
    edge_density_binary
]

plt.figure(figsize=(10, 7))

bars = plt.bar(
    representations,
    edge_values,
    color=[
        "royalblue",
        "seagreen",
        "tomato"
    ],
    edgecolor="black"
)

plt.title(
    "Edge Density Comparison",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel(
    "Image Representation",
    fontsize=13
)

plt.ylabel(
    "Edge Density (%)",
    fontsize=13
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.4
)

for bar, value in zip(
    bars,
    edge_values
):
    plt.text(
        bar.get_x() +
        bar.get_width() / 2,
        bar.get_height() + 0.2,
        f"{value:.2f}%",
        ha="center",
        va="bottom",
        fontsize=12,
        fontweight="bold"
    )

plt.tight_layout()

```

Enal
noti
to g
exec

```

plt.show()

# =====
# 19. IMAGE INFORMATION / PIXEL STATISTICS
# =====

print("\n")
print("=" * 80)
print("                IMAGE REPRESENTATION STATISTICS")
print("=" * 80)

print(
    "\nOriginal Color Image:"
)

print(
    "    Channels      : 3"
)

print(
    f"    Pixel Values   : 0 - 255"
)

print(
    "\nGrayscale Image:"
)

print(
    "    Channels      : 1"
)

print(
    "    Pixel Values   : 0 - 255"
)

print(
    "\nBinary Image:"
)

print(
    "    Channels      : 1"
)

print(
    "    Pixel Values   : 0 and 255"
)

# =====
# 20. ANALYSIS
# =====

print("\n")
print("=" * 80)
print("                ANALYSIS")
print("=" * 80)

print(
    "\n1. ORIGINAL COLOR REPRESENTATION"
)

print(
    "The original color image contains complete color information "
    "and is useful when color plays an important role in the "
    "computer vision application."
)

print(
    "\n2. GRAYSCALE REPRESENTATION"
)

print(

```

Enal
noti
to g
exec

```

    grayscale conversion reduces the image from three color
    "channels to one intensity channel."
)

print(
    "It reduces computational complexity while preserving most "
    "important structural and intensity information."
)

print(
    "\n3. BINARY REPRESENTATION"
)

print(
    f"Binary representation converts pixels into two classes "
    f"using threshold T = {THRESHOLD}."
)

print(
    "It is useful for segmentation, shape analysis, document "
    "processing and simple object detection."
)

print(
    "\n4. EDGE DETECTION"
)

print(
    f"Original edge density : {edge_density_original:.2f}%"
)

print(
    f"Grayscale edge density: {edge_density_gray:.2f}%"
)

print(
    f"Binary edge density   : {edge_density_binary:.2f}%"
)

print(
    "\n5. FEATURE EXTRACTION"
)

print(
    f"Original ORB keypoints : {num_original}"
)

print(
    f"Grayscale ORB keypoints: {num_gray}"
)

print(
    f"Binary ORB keypoints   : {num_binary}"
)

print(
    "\nThe grayscale representation generally preserves structural "
    "information better than binary representation."
)

print(
    "Binary conversion may remove gradient and intensity information, "
    "which can reduce the number of useful feature points."
)

# =====
# 21. FIND BEST REPRESENTATION
# =====

keypoint_results = {

    "Original Color": num_original,

```

Enal
noti
to g
exec

```

    "Grayscale": num_gray,

    "Binary": num_binary
}

best_representation = max(
    keypoint_results,
    key=keypoint_results.get
)

print("\n")
print("=" * 80)
print("                                BEST REPRESENTATION")
print("=" * 80)

print(
    f"\nHighest ORB keypoint count: "
    f"{best_representation}"
)

print(
    f"Number of keypoints: "
    f"{keypoint_results[best_representation]}"
)

# =====
# 22. OBSERVATIONS
# =====

print("\n")
print("=" * 80)
print("                                OBSERVATIONS")
print("=" * 80)

print(
    "\n• Grayscale conversion reduces computational complexity "
    "while preserving important structural information."
)

print(
    "• Binary representation is sensitive to the selected "
    "threshold value."
)

print(
    "• Important details may be lost during binary conversion."
)

print(
    "• Canny edge detection works effectively on grayscale images."
)

print(
    "• Binary images provide simplified boundaries but may "
    "lose gradient information."
)

print(
    "• ORB feature extraction generally detects more meaningful "
    "features when sufficient intensity and structural information "
    "is preserved."
)

# =====
# 23. CONCLUSION
# =====

print("\n")
print("=" * 80)
print("                                CONCLUSION")
print("=" * 80)

print(

```

Enal
noti
to g
exec

```

    "\nimage representation plays a crucial role in computer "
    "vision applications."
)

print(
    "The original color representation preserves the maximum "
    "amount of visual information but requires higher computation."
)

print(
    "Grayscale representation provides a good balance between "
    "image information and computational efficiency."
)

print(
    "Binary representation is computationally efficient and "
    "useful for simple shape-based and segmentation tasks, "
    "but it may lose important intensity and gradient information."
)

print(
    "Therefore, grayscale representation is generally a suitable "
    "choice for edge detection and feature extraction when a "
    "balance between accuracy and efficiency is required."
)

# =====
# 24. OVERALL RESULT
# =====

print("\n")
print("=" * 80)
print("                                OVERALL RESULT")
print("=" * 80)

print(
    "\nGrayscale representation provides a good trade-off between "
    "performance and efficiency, while binary representation is "
    "useful for simpler and faster computer vision applications."
)

# =====
# 25. SAVE COMPARISON TABLE
# =====

df.to_csv(
    "image_representation_comparison.csv",
    index=False
)

print(
    "\nComparison table saved as:"
)

print(
    "image_representation_comparison.csv"
)

# =====
# 26. SAVE OUTPUT IMAGES
# =====

cv2.imwrite(
    "grayscale_image.png",
    gray
)

cv2.imwrite(
    "binary_image.png",
    binary
)

cv2.imwrite(
    "edge_original.png",
    edge_original

```

Enal
noti
to g
exec

```

)

cv2.imwrite(
    "edge_grayscale.png",
    edge_gray
)

cv2.imwrite(
    "edge_binary.png",
    edge_binary
)

cv2.imwrite(
    "orb_original.png",
    cv2.cvtColor(
        orb_original_img,
        cv2.COLOR_RGB2BGR
    )
)

cv2.imwrite(
    "orb_grayscale.png",
    orb_gray_img
)

cv2.imwrite(
    "orb_binary.png",
    orb_binary_img
)

# =====
# 27. FINAL MESSAGE
# =====

print("\n")
print("=" * 80)
print("          EXPERIMENT COMPLETED SUCCESSFULLY")
print("=" * 80)

print(
    "\nGenerated outputs:"
)

print(
    "✓ Original Color Image"
)

print(
    "✓ Grayscale Image"
)

print(
    "✓ Binary Image"
)

print(
    "✓ Canny Edge - Original"
)

print(
    "✓ Canny Edge - Grayscale"
)

print(
    "✓ Canny Edge - Binary"
)

print(
    "✓ ORB Keypoints - Original"
)

print(
    "✓ ORB Keypoints - Grayscale"
)

print(

```

Enal
noti
to g
exec

```
        "\✓ ORB Keypoints - Binary"
    )

    print(
        "\✓ Comparison Table"
    )

    print(
        "\✓ Keypoint Comparison Graph"
    )

    print(
        "\✓ Edge Density Graph"
    )

    print(
        "\✓ Analysis and Conclusion"
    )

    print(
        "\✓ CSV Result File"
    )

    print("\n" + "=" * 80)
```

Enal
noti
to g
exec

Enal
noti
to g
exec

=====

IMAGE REPRESENTATION STUDY

=====

Upload your input image:

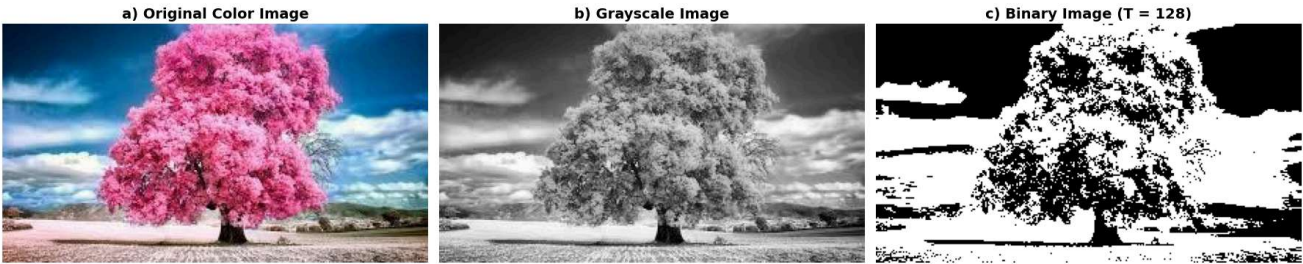
Choose Files

 IMAGE.jpg
IMAGE.jpg(image/jpeg) - 15042 bytes, last modified: 7/26/2026 - 100% done
Saving IMAGE.jpg to IMAGE (1).jpg

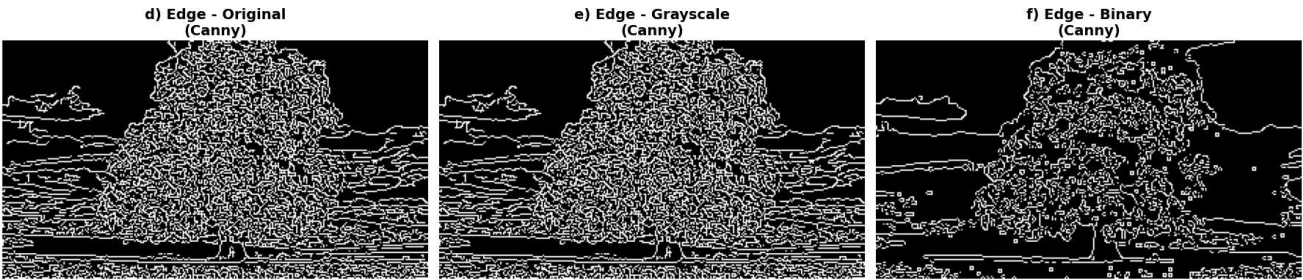
Image loaded successfully!
Image Name : IMAGE (1).jpg
Image Size : (168, 300, 3)

Enal
noti
to g
exec

Image Representation Conversion



Canny Edge Detection Comparison



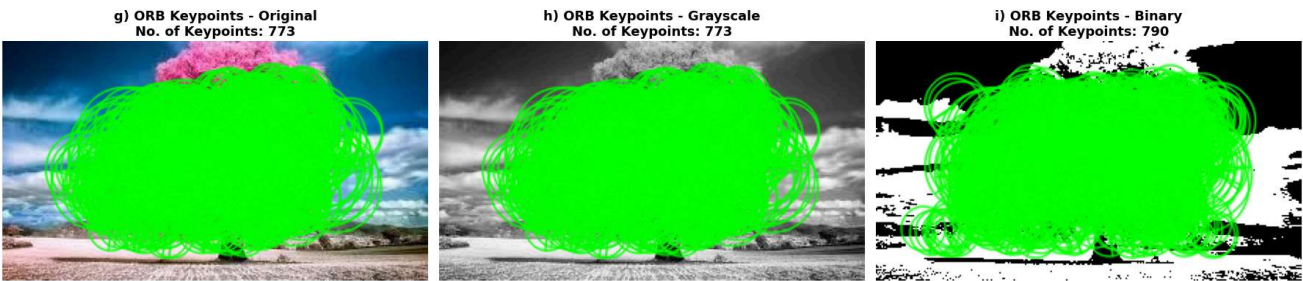
=====

ORB FEATURE EXTRACTION

=====

Original Color Image : 773 keypoints
Grayscale Image : 773 keypoints
Binary Image : 790 keypoints

ORB Feature Extraction



=====

FEATURE EXTRACTION SUMMARY

=====

Original Color Image:
ORB Keypoints = 773

Grayscale Image:
ORB Keypoints = 773

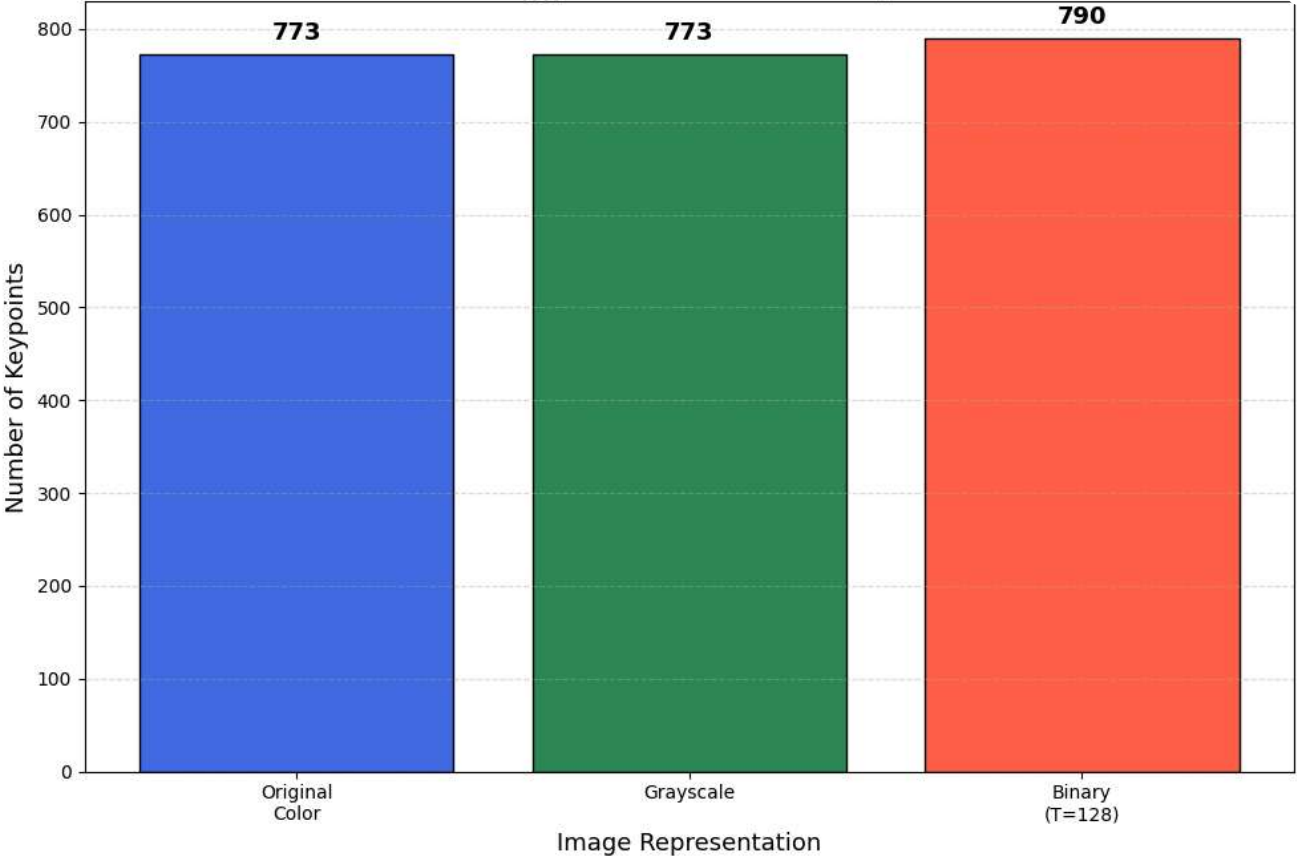
Binary Image:
ORB Keypoints = 790

COMPARISON TABLE						
	Representation	Image Type	Edge Density (%)	No. of Keypoints (ORB)	Computation Complexity	Performance
0	Original Color	Color (BGR)	25.33	773	High	High (Rich Info)
1	Grayscale	8-bit Gray	25.33	773	Medium	Good (Balanced)
2	Binary (T=128)	Binary (0/1)	13.80	790	Low	Moderate (Less Info)

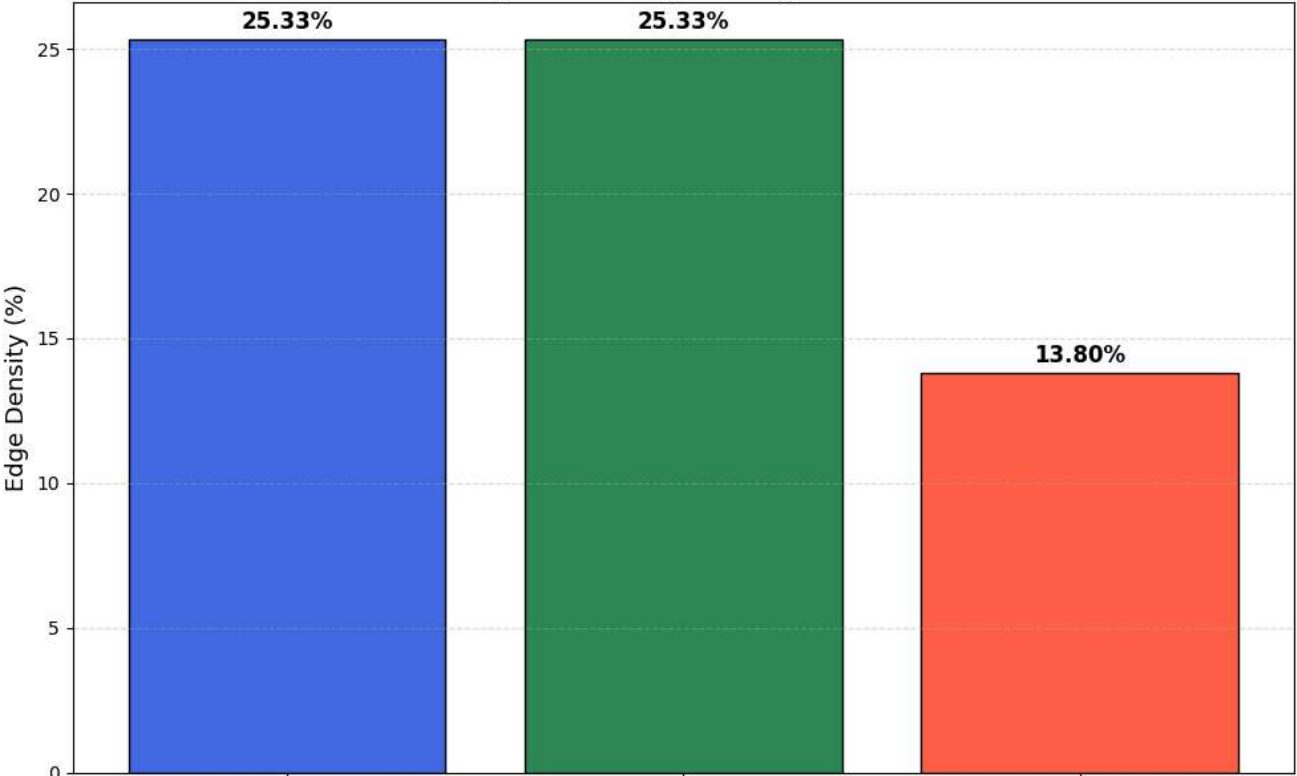


Enal
noti
to g
exerc

No. of Keypoints (ORB) Comparison



Edge Density Comparison



Original
Color

Grayscale

Binary
(T=128)

Image Representation

=====

IMAGE REPRESENTATION STATISTICS

=====

Original Color Image:
Channels : 3
Pixel Values : 0 - 255

Grayscale Image:
Channels : 1
Pixel Values : 0 - 255

Enal
noti
to g
exec